

# On Tropical Semirings and the Limits of Instruction-Level Parallelism

Alexandra W. Chadwick  
University of Cambridge, UK  
alexandra.chadwick@cl.cam.ac.uk

**Abstract**—Instruction-level parallelism is a crucial aspect of high performance processor design. There have been many previous studies that compute upper bounds on the amount of instruction-level parallelism in programs using mathematical models of idealised program execution. Specifically, one upper bound for parallelism is the number of dynamic instructions in a program’s execution divided by the length of the longest chain of dependant instructions in that program (the critical path). Certainly no real machine that takes at least one cycle per instruction can ever execute the program faster than by executing this critical path sequentially.

Computing the length of the longest chain of dependencies in a program can be expressed as find the longest path in a directed acyclic graph representing dependencies between dynamic instructions. Tropical analysis is a branch of pure mathematics using the tropical semiring (also known as the plus-min/max algebra). It has previously been noted that this form of mathematics has applications to graph problems such as computing the shortest path between nodes in a graph. In this work, we show a specific application of tropical analysis to finding the length of the longest dependency chain in a program, and further show a number of useful properties that make this an efficient representation for such problems.

## I. TROPICAL ANALYSIS

Conceptually, in tropical analysis we replace the role of several familiar elements in equations. Really, there are two forms, one using max and one using min. For example, using max we replace:

- the reals as our domain with the reals extended with  $-\infty$ .
- $+$  with max.
- 0, the identity element of  $+$  with  $-\infty$ , the identity element of max.
- multiplication with  $+$ .
- 1, the identity element of multiplication with 0, the identity element of  $+$ .

These replacements form a semiring<sup>1</sup>; a mathematical structure with many familiar properties that we take for granted in conventional linear algebra. For example the following two equivalences both hold, the first using the conventional multiply-plus ring, the second using the plus-max semiring. Note the structural similarity between the two equations:

$$(a + b) \cdot (1 + c) + 0 = a + b + a \cdot c + b \cdot c$$

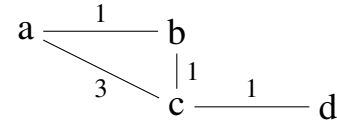
$$\max(\max(a, b) + \max(0, c), -\infty) = \max(a, b, a + c, b + c)$$

<sup>1</sup>It is a semiring, not a full ring, since max has no inverse operator

### A. Shortest paths

It has been noted previously that the min-plus version of this semiring is applicable to the problem of finding the shortest path in a graph. One algorithm to find the shortest path between nodes say  $A$  and  $B$  in a graph is to recursively take the min of all path lengths from all neighbours of  $A$  to  $B$ , plus the cost of the edge from  $A$  to that node. We can intuitively see this will be an equation involving only min and plus operators: exactly those used in the tropical semiring. The clever observation is: if we have an adjacency matrix (with one row and column per node in the graph, the cells in that matrix represent the cost of the edge between the pair of nodes in that row/column,  $\infty$  if no edge) then performing tropical semiring matrix multiplication (i.e. with  $+$  and multiply replaced with min and  $+$ ) of this matrix with itself, corresponds to one step of our shortest path algorithm. Rather delightfully, raising the matrix to power  $\infty$  will converge to the shortest distance matrix for that whole graph.

For example consider the graph:



The adjacency matrix would be:

$$A = \begin{pmatrix} 0 & 1 & 3 & \infty \\ 1 & 0 & 1 & \infty \\ 3 & 1 & 0 & 1 \\ \infty & \infty & 1 & 0 \end{pmatrix}$$

So, in the tropical semiring (using min):

$$A^2 = \begin{pmatrix} 0 & 1 & 2 & 4 \\ 1 & 0 & 1 & 2 \\ 2 & 1 & 0 & 1 \\ 4 & 2 & 1 & 0 \end{pmatrix}$$

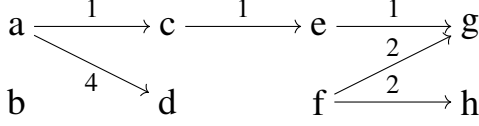
$$A^3 = \begin{pmatrix} 0 & 1 & 2 & 3 \\ 1 & 0 & 1 & 2 \\ 2 & 1 & 0 & 1 \\ 3 & 2 & 1 & 0 \end{pmatrix}$$

$$= A^4 = A^5 = \dots = A^\infty$$

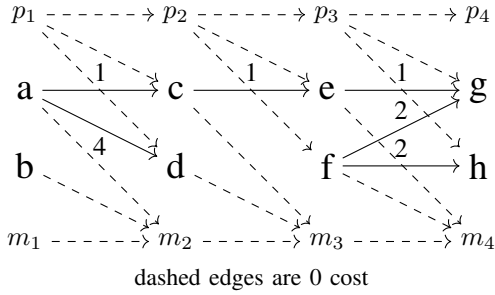
This corresponds to the shortest distance matrix between all pairs of nodes.

### B. Longest paths in directed acyclic graphs

For this work, we instead focus on longest paths in directed acyclic graphs. Consider the following graph:



This graph has a structure that is more typical of the graphs in our research, namely; that the graph is long but not wide. Whilst we could still use an adjacency matrix representation, the structure lends itself to another representation for longest path problems. First of all, we modify the graph such that the longest path will always be connected to the rightmost part of the graph, by adding an extra set of nodes with cost-0 edges, and also so that each path is connected to the first column similarly.



In this new graph, we can see structurally that every single path in the graph is connected to both the first and the final column. Now we can represent the graph with one matrix per adjacent-column pair, encoding the connectivity between those two columns. For example, the connectivity between the first two columns is encoded as:

$$A = \begin{pmatrix} 0 & 0 & 0 & -\infty \\ -\infty & 1 & 4 & 0 \\ -\infty & -\infty & -\infty & 0 \\ -\infty & -\infty & -\infty & 0 \end{pmatrix}$$

where the  $m$ th row and  $n$ th column of this matrix represent the connection from the  $m$ th node in the first column of the graph to the  $n$ th node in the second column of the graph.

Our remaining two columns are represent as

$$B = \begin{pmatrix} 0 & 0 & 0 & -\infty \\ -\infty & 1 & -\infty & 0 \\ -\infty & -\infty & -\infty & 0 \\ -\infty & -\infty & -\infty & 0 \end{pmatrix}$$

$$C = \begin{pmatrix} 0 & 0 & 0 & -\infty \\ -\infty & 1 & -\infty & 0 \\ -\infty & 2 & 2 & 0 \\ -\infty & -\infty & -\infty & 0 \end{pmatrix}$$

These matrices are each  $4 \times 4$  whereas a full adjacency matrix for the original graph would be  $7 \times 7$ . Despite our additional nodes, this means these column-wise matrices have less storage overall to encode compared with the original adjacency matrix, and this trade-off becomes increasingly favourable for column-wise matrices as the graph gets longer.

In the tropical semiring (max), computing the product

$$C \cdot B \cdot A = \begin{pmatrix} 0 & 2 & 2 & 1 \\ -\infty & 3 & -\infty & 4 \\ -\infty & -\infty & -\infty & 0 \\ -\infty & -\infty & -\infty & 0 \end{pmatrix}$$

yields a matrix representing the longest path from the  $m$ th node in the first column to the  $n$ th node in the final column of the graph. Because we have deliberately ensured every path in the graph connects to the first and last column, the largest individual value in this matrix is the longest path in the graph, namely the length 4 path connecting from  $a$  in the first column to  $m_4$  in the final column (via  $d$  and  $m_3$ ).

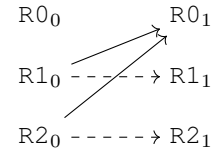
## II. LIMITS OF INSTRUCTION-LEVEL PARALLELISM

In our research, we seek to compute an upper bound for the available instruction level parallelism in a program execution by computing the length of the longest chain of dependent dynamic instructions in that execution. Assuming no processor can ever execute any instruction faster the one cycle, the length of this longest chain represents the minimum execution time of the program on any processor, and hence the number of instructions in the program divided by this chain length is an upper bound for the average number of instructions that could execute in parallel on each cycle. The real amount of instruction-level parallelism that a processor could achieve will always be less than this number if the processor cannot execute any instruction faster than in one cycle.

### A. Register-only longest chains

Ignoring memory dependencies, we could choose to model the program execution's dependency structure as a graph with one node per register per dynamic instruction. Note that this is trillions of nodes for even modest realistic programs. However, similar to the example graph in section I-B, we can organise the graph as a column of nodes per dynamic instruction. The effects of any given instruction are to read some subset of the registers in order to write some others, leaving all other registers unchanged. We can represent this in the graph by having cost-0 edges between unaffected registers, and cost-1 edges from all read registers to all written registers.

For example, on a machine with three registers, we might represent the instruction  $R0 = \text{ADD } R1, R2$  as the graph:



Using the technique described in section I-B we can extend the graph and obtain a matrix per dynamic instruction, and matrix multiplication of these matrices encodes the effect of sequentially executing those instructions for the purpose of critical-path length calculation. On real architectures, for almost all static instructions the dynamic execution of that static instruction always results in the same matrix, because the dependency structure of almost all instructions is a static

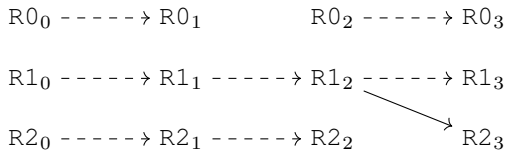
property. Hence, these matrices are frequently reused as the number of static instructions in a program is vastly smaller than the number of dynamic instances of those instructions. Consider for example a loop of 2 static instructions that loops 16 times. If we have matrix  $A$  representing the first instruction and matrix  $B$  representing the second, then  $B \cdot A$  represents one iteration of the loop,  $(B \cdot A)^{16}$  represents all sixteen iterations. This can be compute efficiently using just 5 matrix multiplications, which is fewer than the 32 instructions that dynamically executed. In this way, tropical semiring-based matrix representation allows us to beat a naive linear implementation of dependency length tracking that iterates by instruction.

An additional useful property is that this representation allows us to simultaneously compute the longest paths in subregions of interest. For example, as well as computing the longest dependency chain in a whole program execution, we could also compute the same for each function invocation in the program.

### B. Through-memory dependencies

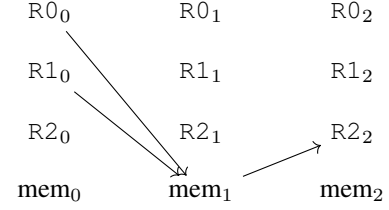
In real programs, the true critical path often involves memory operations. Here, a value will be copied to memory from a register by a store instruction, and then potentially millions of instructions later that memory location can be read by a load instruction. A simple naive approach to model this would be, similar to registers, to introduce one node per instruction for every location in memory. This however results in a catastrophic explosion of size; real programs often use millions of memory locations, so the graph size would increase by many orders of magnitude. It may nevertheless be possible to work with this representation; the matrices that represent each instruction would now be enormous, but would be incredibly sparse as each instruction is only able to read or write two or three values. Hence the matrices will mostly be a long diagonal of zeros, with a handful of non-zero entries. A carefully chosen matrix representation could avoid storing the long diagonal of zeroes, and instead only store the non-default entries.

An alternative approach, currently used in our work, is to model the effect of loads and stores using tropical semiring matrix *addition* rather than just multiplication. Consider a simple program like `STORE R0 at R1; R0 = 1; R2 = LOAD from R1`. If we have a matrix  $A_0$  representing the store (without considering memory),  $B$  representing the second instruction, and  $C_0$  representing the load (without considering memory) then  $C_0 \cdot B \cdot A_0$  represents the register-only longest dependency chains. i.e.



We can augment this with a matrix  $A_1$  which represents the action of the store copying a value to ‘memory’ (a single extra node in the graph) and a matrix  $C_1$  which represents the

action of the load copying the value from memory (that same extra node) to a register. i.e.



Now  $C_1 \cdot A_1$  represents the through-memory dependencies of this program. So  $C_0 \cdot B \cdot A_0 + C_1 \cdot A_1$  represents all possible dependencies of this program. Crucially, if instructions preceed or proceed this part of the program, we can just multiply as normal, so for example if the next instruction is represented by matrix  $D$ , then the whole program would be represented by  $D \cdot (C_0 \cdot B \cdot A_0 + C_1 \cdot A_1)$

This matrix addition-based approach allows the tropical semiring matrix representation to extend to long-range through memory dependencies by injecting additions into the equation whenever a dependent load/store pair is encountered. From an implementation perspective, this unfortunately means tracking each store in the program until all loads are known in order to form these pairs, and sadly loads and stores are not necessarily well-bracketed, so the resulting matrix equations can get quite convoluted with many extra matrix operations. It also unfortunately ruins the composability of the original representation; the matrix representing the longest path through a subgraph does not necessarily usefully appear as a term in the equation representing the whole graph due to matrix additions.

It is therefore an outstanding research problem for us to more efficiently encapsulate loads and stores into the tropical semiring-based methodology for critical path length computation.